Convenient Algebraic Programming with Coercive Subtyping

Henry Blanchette[1*] and Aaron Stump[2*]

[1]University of Maryland.
[2]Boston College.

*Corresponding author(s). E-mail(s): blancheh@umd.edu;
stumpaa@bc.edu;

Abstract

This paper presents a system for more convenient algebraic programming, a form of total functional programming where datatypes are defined as least fixed points of covariant signature functors, and recursive functions are defined as algebras. This approach requires the use of certain combinators to roll and unroll fixed points, and to convert algebras to functions which can be applied to recurse over data. These combinators are a burden for programming in this style. To alleviate that burden, in this paper we propose to insert them automatically, using coercive subtyping. The type checker generates subtyping constraints, which are then solved with the help of a custom logic programming engined called Chronolog. The subtyping axioms are given to Chronolog as rules, which uses them to solve the subtyping constraints. To avoid infinite applications of rules for unrolling signature functors, Chronolog provides a mechanism for suspending and resuming certain forms of goals. The subtyping axioms presented to Chronolog are an algorithmic version of the usual declarative rules for algebraic programming. We prove equivalence of the declarative and algorithmic rules, in Agda. We present numerous examples of algebraic programs written in our system.

Keywords: strong functional programming, type-based termination, Mendler-style recursion

1 Introduction

Strong functional programming is a form of pure functional programming where all functions are statically required to terminate on all inputs. One way to achieve this is

through an algebraic approach: datatypes are least fixed points μF of signature functors F, and recursive functions are least fixed points $(\!|a|\!)$ of algebras a. The functions are called catamorphisms. The signature functor can be seen as describing one layer of the data. The least fixed point then describes data consisting of any finite number of layers. We may unroll μF to $F\ (\mu F)$, and roll it back again. Such transformations are needed to allow construction of data by applying constructors of the signature functor, and destruction by pattern-matching against those constructors. Algebras interpret the operations declared for one layer of the data, and catamorphisms then apply algebras across all the layers. There is no other mechanism for recursion or looping in the language. All functions written in this style are guaranteed to terminate.

One hindrance to algebraic programming is the need to apply functions for rolling and unrolling signature functors, and to apply $(\!|-|\!)$ to obtain a recursive function from an algebra.

2 Motivating example

3 Syntax

4 Constraint-Generating Typing

- $\mathscr{D}$ is the set of datatype definitions, which are pairs of $\langle D, V \rangle$ where D is an identifier and V is a set of constructor signatures.
- The meta-function $\mathsf{freshenFunctorParam}(A)$ replaces all instances of the functor parameter type in A with a fresh type variable.
- The meta-function $\mathsf{typeOfCoercion}(c)$ calculates the pair of the domain and codomain of a coercion.

$$\textsc{InferVar}\quad \frac{(x : A) \in \Gamma}{\Gamma \vdash x : A \mid \varnothing}$$

$$\textsc{InferCon}\quad \frac{\langle D, V\rangle \in \mathscr{D} \quad \langle c, \langle \dots A'_i, \dots \rangle\rangle \in V \quad (\forall i)\ \Gamma \vdash a_i : A_i \mid \mathcal{C}_{a_i}}{\Gamma \vdash D.c(\dots a_i, \dots) \mid \bigcup_i (\mathcal{C}_{a_i} \cup \{A_i <: \mathsf{freshenFunctorParam}(A'_i)\})}$$

$$\textsc{InferLam}\quad \frac{\Gamma, (x : A) \vdash b : B \mid \mathcal{C}_b}{\Gamma \vdash \lambda(x : A).b : A \to B \mid \mathcal{C}_b}$$

$$\textsc{InferApp}\quad \frac{A', B \text{ are fresh type variables} \quad \Gamma \vdash f : F \mid \mathcal{C}_f \quad \Gamma \vdash a : A \mid \mathcal{C}_a}{\Gamma \vdash (f\ a) : B \mid \mathcal{C}_f\{F <: A' \to B\} \cup \mathcal{C}_a \cup \{A <: A'\}}$$

$$\textsc{InferGamma}\quad \frac{\langle D, V\rangle \in \mathscr{D} \quad (\forall i)\ \langle c_i, \langle \dots A'_{ij}, \dots \rangle\rangle \in V \quad (\forall i)\ \Gamma, \dots x_{ij} : A'_{ij} \cdots \vdash b_i : B_i \mid \mathcal{C}_{b_i}}{\Gamma \vdash \gamma(D, B)a.\{\dots\ c_i(\dots x_{ij}, \dots) \mapsto b_i \mid\ \dots\} \mid \bigcup_i (\mathcal{C}_{b_i} \cup \{B_i <: B\})}$$

$$\textsc{InferOmega}\quad \frac{\Gamma, (R : *), (f : R \to A), (x : D\ R) \vdash b : B \mid \mathcal{C}_b}{\Gamma \vdash \omega(D, B, R, f, x).b : D \Rightarrow B \mid \mathcal{C}_b}$$

$$\textsc{InferLet}\quad \frac{\Gamma \vdash a : A \mid \mathcal{C}_a \quad \Gamma, (x : A') \vdash b : B \mid \mathcal{C}_b}{\Gamma \vdash \text{let } x : A' = a \text{ in } b : B \mid \mathcal{C}_a \cup \{A <: A'\} \cup \mathcal{C}_a}$$

$$\textsc{InferCoerce}\quad \frac{\langle A', B\rangle := \mathsf{typeOfCoercion}(c) \quad \Gamma \vdash a : A \mid \mathcal{C}_a}{\Gamma \vdash \text{coerce } c\ a : B \mid \mathcal{C}_a \cup \{A <: A'\}}$$

Fig. 1 Subtype constraint generation rules.

5 Subtyping

5.1 Declarative

$$\textsc{SubData}\ \frac{\langle D, \_ \rangle \in \mathscr{D}}{D <: D} \qquad \textsc{Omega}\ \frac{\text{TODO: require that R is an omega var}}{R <: R} \qquad \textsc{Arr}\ \frac{A' <: A \quad B <: B'}{A \to B <: A' \to B'}$$

$$\textsc{Cov}\ \frac{A <: A'}{D\ A <: D\ A'} \qquad \textsc{Alg}\ \frac{A <: A'}{D \Rightarrow A <: D \Rightarrow A'} \qquad \textsc{Roll}\ \frac{}{D\ D <: D} \qquad \textsc{Unroll}\ \frac{}{D <: D\ D}$$

$$\textsc{Cata}\ \frac{}{D \Rightarrow A <: D \to A} \qquad \textsc{Trans}\ \frac{A <: A' \quad A' < A''}{A <: A''}$$

Fig. 2 Declarative subtyping rules.

5.2 Algorithmic

$$\textsc{DecSubData}\ \frac{\langle D, \_ \rangle \in \mathscr{D}}{D <: D} \qquad \textsc{DecOmega}\ \frac{\text{TODO: require that R is an omega var}}{R <: R}$$

$$\textsc{DecArr}\ \frac{A' <: A \quad B <: B'}{A \to B <: A' \to B'} \qquad \textsc{DecCov}\ \frac{A <: A'}{D\ A <: D\ A'} \qquad \textsc{DecAlg}\ \frac{A <: A'}{D \Rightarrow A <: D \Rightarrow A'} \qquad \textsc{DecRoll}\ \frac{A <: D}{D\ A <: D}$$

$$\textsc{DecUnroll}\ \frac{D <: A}{D <: D\ A} \qquad \textsc{DecCata}\ \frac{B <: D \quad A <: C}{D \Rightarrow A <: B \to C}$$

Fig. 3 Algorithmic subtyping rules.

5.3 Equivalence of declarative and algorithmic subtyping

6 Implementation with Chronolog

7 Returning to the Examples

8 Related Work

Turner introduced the term *strong functional programming*, presented several arguments in favor of the approach, and proposed an *elementary* way to realize such a language, in the form of static checks for structural recursion [1]. Mendler proposed a recursor using an abstract type to ensure that recursive calls are permitted only on subdata for an inductive type [2].

9 Conclusion and future work

References

[1] Turner, D.A.: Elementary Strong Functional Programming. In: Proceedings of the First International Symposium on Functional Programming Languages in Education. FPLE '95, pp. 1–13. Springer, Berlin, Heidelberg (1995)

[2] Mendler, N.P.: Inductive types and type constraints in the second-order lambda calculus. Annals of Pure and Applied Logic **51**(1), 159–172 (1991)